FracVal python wrapper notes

Friday, November 13, 2026 11:38 AM

Overlap Model:

The overlap model is quite simple and just adjusts the distance for particle-particle contact when the second subcluster is placed.

$d_{ij} = (R_i + R_j)(1 - G)$ G = overlap fraction

Two modes:

1) Fixed overlap - every point contact uses the same overlap fraction

2) Statistical - G is sampled from a normal distribution so each particle contact does not have the same overlap.

↳ This is applied locally per each contact point so it does not affect other points in the aggregate (i.e. when we do our checks for overlap, of other particles in this configuration, we still use R1+R2)

↳ Also note that the overlap can cause changes to the derived geometry like radius of gyration.

For this paper w/ the polydisperse case, $r_{p,g}$ is supplied but it is not fully known if the target Rg is calculated using the exact input number directly or the geometric mean recomputed from the specific particles actually sampled. *Needs checking

Rg for actual generated aggregate

$$\mathbf{r}_{\mathrm{cm}} = \frac{\sum_i r_i^3 \mathbf{r}_i}{\sum_i r_i^3}. \tag{6.1}$$

The reported radius of gyration includes the intrinsic moment of each solid sphere:

$$R_g = \left[\frac{\sum_i r_i^3 \left(\|\mathbf{r}_i - \mathbf{r}_{\mathrm{cm}}\|^2 + \frac{3}{5} r_i^2 \right)}{\sum_i r_i^3} \right]^{1/2}. \tag{6.2}$$

The bounding radius is the maximum center distance from the aggregate center of mass plus that particle's radius.

9.1 Fractal target relation

The legacy placement routines compute target gyration radii using the geometric mean primary-particle radius and the fractal relation implemented in the source:

$$R_{g,\mathrm{target}} = r_{p,g} \left(\frac{N}{k_f} \right)^{1/D_f}. \tag{9.1}$$

For polydisperse working sets, $r_{p,g}$ is obtained from the geometric mean of the radii currently being considered. This relation guides PCA/CCA placement; the Python post-generation `Aggregate.radius_of_gyration` is an independently calculated geometric metric of the final spheres.

Chapter 17

Command and API Reference

17.1 Installed commands

Command	Purpose
`fracval-gui`	Launch native PySide6 desktop application.
`fracval-python`	Generate one aggregate from command-line Python arguments.
`fracval-info`	Report Python, package, extension and backend diagnostics.
`fracval-qt-check`	Report Qt/PySide6 plugin paths and platform availability.

17.2 Public Python functions

Name	Purpose
`generate(config, ...)`	Generate one aggregate.
`generate_batch(count, config, ...)`	Return a list of aggregates.
`iter_generate_batch(count, config, ...)`	Stream aggregates one at a time.
`available_backends()`	List currently usable generation backends.
`extension_available()`	Test whether the F2PY extension imports.
`load_data(path)`	Read a four-column native DAT file into NumPy.
`save_bundle(aggregate, directory, ...)`	Save data, contacts, and metadata with a common stem.
`load_bundle(metadata_path)`	Reconstruct an Aggregate from a saved bundle.
`plot_3d(...)`	Return an interactive Plotly 3-D figure.
`plot_static(...)`	Return a Matplotlib 3-D figure.

`runtime_info()`	Return environment diagnostics as a dictionary.

17.3 FracVALConfig fields

Field	Default	Meaning
`n`	100	primary-particle count
`df`	1.79	fractal dimension
`kf`	1.40	fractal prefactor
`rp_g`	15.0	geometric mean radius
`rp_gstd`	1.0	geometric radius standard deviation
`ext_case`	0	legacy extreme-case flag
`nsubcl_perc`	0.10	sub-cluster fraction
`tol_ov`	10^{-8}	unintended-overlap numerical tolerance
`seed`	None	fixed seed or Python-selected fresh seed
`max_attempts`	250	maximum construction restarts in extension path
`overlap_mode`	none	none, fixed, or statistical
`overlap_fraction`	0.0	fixed intended-contact overlap
`overlap_mean`	0.05	statistical mean
`overlap_std`	0.02	statistical standard deviation
`overlap_max`	0.12	statistical hard upper bound

17.4 ViewerAppearance fields

Field	Default	Meaning
`color_mode`	solid	solid or radius-based coloring
`particle_color`	#4C78A8	solid particle color
`colorscale`	Viridis	Plotly radius color scale
`opacity`	0.96	particle opacity
`shininess`	0.55	specular/gloss appearance in sphere mode
`background_color`	#FFFFFF	scene/page background
`show_axes`	false	show XYZ axes/grid
`show_colorbar`	false	show radius legend when radius-colored
`show_title`	true	show figure title

Appendix B

Recommended Scientific Workflow Checklist

1. Record `N`, `Df`, `kf`, particle-size distribution, overlap model, and seed.
2. Use a fixed seed while developing or validating a case.
3. Run `make test` after modifying the Fortran/Python interface.
4. Save aggregate metadata or a bundle, not only a DAT file.
5. Keep overlap distinct from `tol_ov`.
6. Use the F2PY backend for normal Python work and the executable backend as a useful cross-check.
7. For parameter sweeps, log failed seeds/configurations instead of silently skipping them.
8. Use multiprocessing rather than threads for process-level generation parallelism.
9. Rebuild the native extension after changing Python versions or architectures.
10. Keep the GUI as an interface layer; add reusable scientific functionality to the public Python API.

Recommended practice

The fastest route for a new user is: create a virtual environment, run `make install-gui PYTHON=python`, run `make`, verify with `fracval-info`, and then use either `fracval-gui` or the public `fracval` Python API. The full project is designed so those two workflows share the same Fortran generator.